

This project covered the development of an infinite sidescroller, where the “level” chunks are generated procedurally.

ALGORITHM

The basic idea is each session is composed of a number of level “chunks”. At any given time, three chunks are always present in the play “area”. Each chunk comprises of 8 “gates” – 4 entry gates and 4 exit gates. These gates are the core of the algorithm and are created as Scriptable Objects, with three distinct properties:

1. Prefab – Type = List<GameObject>
2. Entry Gate – Type = Enum
3. Exit Gate – Type = Enum

Each gate enumerator contains four gates labelled “One, Two, Three, Four” (to keep it simple). The next step is where the algorithm generates the first chunk, which is defined by the developer (so that there exists an initial value for exit gate). After that, the algorithm takes the previous exit gate value, and searches within the list of scriptable objects in the LevelGenerator.cs script, for entries that have the same value of entry gates. Once it finds a scriptable object entry, it adds it to an “AllowedChunkList”. Once all the entries have been found, a randomizer picks the next chunk in sequence from this AllowedChunkList, up until the integer value that specifies the maximum number of level chunks that need to exist simultaneously. In each chunk, there exists an “Exit Trigger” which is of type Box2D collider. Once the player enters this trigger zone, the above selection of chunks takes place, and after a delay of 1 second (float), the last chunk in the three chunk sequence gets destroyed. This saves up on space and processing time.

PSEUDOCODE

1. Load all chunk ScriptableObjects (SO) into LevelGeneration.cs
2. On Play, instantiate first_SO
3. Once first SO is instantiated, find exit gate of first_SO and set it as previousExitGate
4. if(previousExitGate == entry_gate of any SO in LevelGeneration.cs)
 Add SO to List<ScriptableObjects> allowedChunks
5. From allowedChunks randomly pick one and instantiate till total allowed_session_chunks value is reached
6. Check if player entered the exit trigger of a chunk
 if (yes) then destroy first chunk in allowed_session_chunks AND add a new chunk based on steps 3 to 5

GAME LOOP

For its current state, the game does not have a proper gameplay design. However, the timing of destruction of the chunk has been set in such a way, that it mimics as if the player is escaping a crumbling world (behind them) and their objective is to always move forward.

If the player falls down (a.k.a. dies), the scene gets reset and a new set of chunks are instantiated.

FUTURE WORK

The plan is to expand this project into a full-fledged game and include future assignments in it (such as optimization) within the game loop. Also, the plan is to contact a few other graduate students, to provide the art direction for it.

ACKNOWLEDGEMENTS

The player controller is an advanced script, that has been provided by Matthew Spencer (a.k.a Taro Dev on Youtube). Link to the full 2D Player Controller package [here](#). These assets are free to download and use in one's work.

The initial idea for this concept was developed after watching and recreating Matt MirrorFish's tutorial on creating an endless racetrack using procedural generation concepts. The full playlist can be found [here](#). All of the scripts for this assignment have been created from scratch, and only the concept was understood from the original.

IMAGES

